Snowflake Cortex Code (CoCo)

Technical Project Documentation

Apple Retail Sales Data Platform

Medallion Architecture | Dynamic Tables | SCD Type 2 | Cortex Analyst

Choudhury Debashis Dash

Data Engineer | ETL/ELT Developer

Snowflake Certified — Data Engineering Professional | debashisdash1999@gmail.com

1. Project Overview

This document describes the end-to-end design and implementation of a cloud data platform built entirely on Snowflake, using an Apple Retail Sales domain as the data source. The project was developed with the assistance of Snowflake Cortex Code (CoCo) — an AI assistant built into Snowflake — which was used to generate SQL, DDL statements, and pipeline logic throughout the project.

The platform takes raw CSV files containing sales data, processes them through three structured layers — Bronze, Silver, and Gold — applies data quality checks, builds an analytical star schema, pre-computes aggregations, and finally exposes the entire dataset to business users through natural language querying, so they can ask questions in plain English without writing a single line of SQL.

1.1 What Problem Does This Project Solve?

In a typical organization, raw data arrives from multiple source systems in messy, unprocessed formats. Before any business question can be answered, engineers must:

- Clean and validate the data
- Transform it into a structured, query-optimized format
- Build relationships between different datasets (customers, products, stores, transactions)
- Make it accessible to non-technical business users

This project demonstrates all four of those steps from start to finish, using Snowflake's modern, cloud-native feature set — and then goes one step further by adding a natural language interface so anyone can query the data just by typing a question.

1.2 Technology Stack

Component	Technology / Tool
Cloud Data Platform	Snowflake
AI Development Tool	Snowflake Cortex Code (CoCo)
File Ingestion	Snowflake Internal Stage + COPY INTO command
Schema Detection	Snowflake INFER_SCHEMA function
Transformation	Dynamic Tables (Snowflake-native, incremental)
Orchestration	Snowflake Task DAG (design reference only)
Data Modeling	Star Schema — SCD Type 2 Dimensions
Natural Language Query	Snowflake Cortex Analyst + YAML Semantic Model
Constraint Management	Informational NOT ENFORCED FK/PK constraints
Version Control	GitHub
Environment Strategy	Dev → QA → Prod (Context-based promotion)

1.3 Project Scale — Data at a Glance

Object	Count / Size
Source CSV files	13 files
Bronze tables	13 (raw landing)
Silver Dynamic Tables	13 (cleaned & validated)
Gold Dimension Tables	5 (DIM_DATE, DIM_CUSTOMER, DIM_STORE, DIM_COUNTRY, DIM_PRODUCT)
Gold Fact Tables	2 (FACT_SALES_HEADER, FACT_SALES_ITEM)
Gold Bridge Table	1 (BRIDGE_PRODUCT_COUNTRY)
Gold Aggregated Tables	3 (Daily, Weekly, Monthly)
Total Gold Objects	11
Customers	50,000 rows
Sales Transactions	86,107 rows (header + item)
Stores	80 rows
Products (SKUs)	53 rows
Countries	5 rows
Date range covered	3,443 unique dates
Product-Country mappings	212 rows (bridge)

2. Data Domain — Apple Retail Sales

The project uses an Apple Retail Sales domain — a realistic, multi-dimensional dataset that mirrors Apple's actual product and business structure. This domain was chosen because it naturally provides all the components needed for a complete data platform: geographic hierarchy, a complex product hierarchy, customer data, store operations, and transactional sales data.

2.1 Why This Domain?

Think of it this way: Apple sells the same product — for example, an iPhone 15 Pro Max 256GB in Natural Titanium — in different countries, through different store formats, to different types of customers, with different tax rates, currencies, and pricing tiers. Capturing all of that complexity in a data platform requires well-designed dimension tables, a product hierarchy, and a geographic model. This is exactly what this project builds.

2.2 Source Data Entities

The source data is organized into five logical groups, each arriving as separate CSV files:

Group	Source Entities (CSV Files)
Geographic Reference	Region Master, Country Master, Currency Master, Tax Master
Product Hierarchy	Product Category, Product Family, Product Model, Product SKU, Product Country Availability
Store Master	Store Master (80 stores across multiple countries)
Customer Master	Customer Master (50,000 customers)
Sales Transactions	Sales Header (transaction-level), Sales Item (line-item level)

2.3 Product Hierarchy Explained

Apple's product structure follows a strict four-level hierarchy. Understanding this is important because it is modelled faithfully in this project:

Level	Example
Category	iPhone
Family	iPhone 15 Series
Model	iPhone 15 Pro Max
SKU	iPhone 15 Pro Max 256GB Natural Titanium

Additionally, not every SKU is available in every country. For example, certain models may launch in the US before they launch in India. This creates a many-to-many relationship between products and countries — which is handled by a dedicated Bridge Table in the Gold layer.

3. Environment Design — Dev, QA, and Production

Before any data loading begins, the first step is to set up the database environments. In a professional data engineering project, you never develop directly in production. Instead, you work through a structured promotion pipeline: Development → Quality Assurance → Production.

3.1 Why Three Environments?

Imagine building a bridge — you would not test your design on the real bridge while traffic is crossing. You test it in a controlled environment first. The same principle applies here:

- Development (Dev) — where all new tables, pipelines, and transformations are built and tested. If something breaks here, it affects nobody.
- Quality Assurance (QA) — where completed work is tested end-to-end before going live. Mirrors Dev exactly. Acts as the final gate before production.
- Production (Prod) — the live environment. Data here is trusted, protected, and used for real business decisions.

3.2 Database Configuration

Setting	Dev & QA	Production
Database Type	TRANSIENT	PERMANENT
Time Travel	1 day (max for transient)	7 days
Fail-safe Storage	None (cost saving)	Full (7-day protection)
Purpose	Development & testing	Live business data

Why TRANSIENT for Dev and QA? Snowflake's TRANSIENT database type does not maintain a Fail-safe period (the extra 7-day storage Snowflake keeps for disaster recovery). Since Dev and QA data is reproducible — we can always re-run the pipeline — there is no reason to pay for Fail-safe storage there. This is standard Snowflake cost optimisation practice.

Why 7-day Time Travel for Production? Time Travel allows you to query or recover data as it existed at any point in the past. 7 days means if someone accidentally drops a table or runs a bad DELETE on a Friday, you have until the following Friday to recover the data. This is the maximum on Snowflake Standard Edition and is a recognised best practice for production environments.

3.3 Schema Structure Within Each Environment

Each of the three databases (SALES_DEV, SALES_QA, SALES_PROD) contains the same four schemas:

Schema	Purpose
BRONZE	Raw data landing zone. Also hosts the Internal Stage where CSV files are uploaded.
SILVER	Cleansed, validated, and deduplicated data. Structured and ready for modeling.
GOLD	Business-ready analytical model — dimension tables, fact tables, and aggregations.
COMMON	Shared utilities used across all layers — file format definitions, sequence objects, stored procedures.

Every database object — tables, Dynamic Tables, file formats — has a meaningful COMMENT attached to it. This was a deliberate design decision: comments make the schema self-documenting, which is critical when multiple engineers work on the same platform, when BI tools scan the schema, and when building the Semantic Layer for Cortex Analyst.

BRONZE LAYER

Raw Data Ingestion — the landing zone for all source data

4. Bronze Layer — Raw Data Ingestion

The Bronze layer is the entry point of the entire pipeline. It's one and only job is to get raw data from CSV files into Snowflake as quickly and reliably as possible, without changing a single value. Think of it as a digital post box — everything that arrives gets stored exactly as it came in.

4.1 File Format Object

Before any files can be loaded, Snowflake needs to know how to read them. A File Format object is created in the COMMON schema to define the rules for parsing CSV files:

Setting	Value / Reason
TYPE	CSV — the source files are comma-separated
FIELD_OPTIONALLY_ENCLOSED_BY	Double quote (") — handles fields that contain commas within them
SKIP_HEADER	1 — ignores the first row of each file (the column header row)

Storing this in the COMMON schema means it is defined once and reused by every COPY INTO statement across all 13 tables — no duplication, single point of maintenance.

4.2 Internal Stage

An Internal Stage (SALES_ANALYTICS_STAGE) is created inside the BRONZE schema. Think of a stage as a secure upload folder inside Snowflake. The 13 CSV files are manually uploaded to this stage via Snowsight (Snowflake's web interface) before the pipeline runs.

The files are organized in sub-folders within the stage to keep source groups separate:

Stage Sub-folder	Files
initial-load/country-master/	region_master.csv, country_master.csv, currency_master.csv, tax_master.csv
initial-load/product-master/	product_category_master.csv, product_family_master.csv, product_model_master.csv, product_sku_master.csv, product_country_availability.csv
initial-load/store-master/	store_master.csv
initial-load/customer-master/	customer_master.csv
initial-load/sales-transaction/	sales_header.csv, sales_item.csv

4.3 Bronze Tables — Structure and Design

Each source entity gets its own Bronze table. All 13 tables follow the same structural pattern:

- Business columns — copied exactly from the source file, no transformation
- Three audit metadata columns — added by the pipeline, not present in the source file

The three audit columns are the most important design element in the Bronze layer:

Audit Column	Source
__FILE_NAME	METADATA$FILENAME
__ROW_NUMBER	METADATA$FILE_ROW_NUMBER
__LOAD_TS	CURRENT_TIMESTAMP()

Why keep raw data untouched? The Bronze layer never modifies, filters, or rejects any data. Every record that arrives — even if it contains errors — is stored as-is. This is intentional: it preserves a complete audit trail of what was received from the source. If the Silver layer's validation logic has a bug, or if business rules change, you can always re-process from Bronze without going back to the source system.

4.4 COPY INTO — The Ingestion Command

Data is loaded from the stage into Bronze tables using Snowflake's COPY INTO command. The key technique used here is referencing the positional columns ($1, $2, $3…) rather than column names — this works with INFER_SCHEMA and allows flexibility if the source file structure changes slightly.

A representative example of the pattern used:

```
COPY INTO SALES_DEV.BRONZE.REGION_MASTER
FROM (
    SELECT $1, $2, $3, $4, $5, $6, $7,
        METADATA$FILENAME,
        METADATA$FILE_ROW_NUMBER,
        CURRENT_TIMESTAMP()
    FROM @SALES_DEV.BRONZE.SALES_ANALYTICS_STAGE/initial-load/country-
master/region_master.csv
)
FILE_FORMAT = SALES_DEV.COMMON.CSV_FORMAT;
```

This same pattern is applied for all 13 Bronze tables, with the column count and file path adjusted per entity.

SILVER LAYER

Data Cleansing, Validation & Deduplication — making the data trustworthy

5. Silver Layer — Cleansing and Transformation

If Bronze is the post box, Silver is the sorting room. By the time data reaches Silver, it has been cleaned, validated, deduplicated, and quality-flagged. The Silver layer answers a fundamental question that Bronze cannot: is this data correct and trustworthy?

5.1 The Technology: Dynamic Tables

Every Silver table is implemented as a Snowflake Dynamic Table — not as a regular table populated by a Stored Procedure or a traditional Task + Stream pattern.

A Dynamic Table is a Snowflake object that automatically maintains itself. You define it with a SQL SELECT query, and Snowflake takes care of keeping it up to date whenever the source data (Bronze) changes. You never need to write code to trigger a refresh — it just happens.

Feature	Dynamic Tables
Incremental processing	Built-in, automatic
Downstream refresh	Cascades automatically
Orchestration code	None needed
Cost	Efficient — processes only changes
Maintenance	Low — SQL only

All 13 Silver Dynamic Tables are configured with:

- TARGET_LAG = DOWNSTREAM — meaning Silver does not refresh on a fixed schedule. It refreshes when something downstream (Gold) needs updated data. This avoids unnecessary compute.
- REFRESH_MODE = INCREMENTAL — meaning only new or changed records from Bronze are processed. The entire Bronze table is not re-scanning every time.
- INITIALIZE = ON_CREATE — the table is populated immediately when it is first created, so data is available right away.
- WAREHOUSE = COMPUTE_WH — the virtual warehouse used for compute during refresh.

5.2 What Happens in Silver? Three Key Transformations

Transformation 1: Deduplication

The same record can arrive in Bronze multiple times — for example, if a CSV file is accidentally uploaded twice, or if an incremental load re-delivers records that were already loaded. Without deduplication, downstream Gold tables would contain duplicate rows, which would make every metric double-counted.

Every Silver Dynamic Table solves this with a QUALIFY window function:

```
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY <business_key>
    ORDER BY __LOAD_TS DESC, __ROW_NUMBER DESC
) = 1
```

This means: for each unique business key, keep only the record with the most recent load timestamp. If the same region code arrives twice, only the latest version survives into Silver. The partition key varies by entity:

Silver Table	Deduplication Key (PARTITION BY)
REGION_MASTER	REGION_CODE
COUNTRY_MASTER	COUNTRY_CODE
CURRENCY_MASTER	CURRENCY_CODE
TAX_MASTER	TAX_CODE
PRODUCT_CATEGORY_MASTER	CATEGORY_CODE
PRODUCT_FAMILY_MASTER	FAMILY_CODE
PRODUCT_MODEL_MASTER	MODEL_CODE
PRODUCT_SKU_MASTER	SKU_CODE
PRODUCT_COUNTRY_AVAILABILITY	SKU_CODE + COUNTRY_CODE (composite — two-column key)
STORE_MASTER	STORE_CODE
CUSTOMER_MASTER	CUSTOMER_ID
SALES_HEADER	TRANSACTION_ID
SALES_ITEM	TRANSACTION_LINE_ID

Composite key for Product Country Availability:
PRODUCT_COUNTRY_AVAILABILITY uses a two-column deduplication key (SKU_CODE + COUNTRY_CODE) because neither column alone is unique — the same SKU can appear in many countries, and the same country can have many SKUs. Only the combination of both is unique.

Transformation 2: Data Quality Validation

Each Silver table includes a derived column called IS_VALID_RECORD (BOOLEAN). This is calculated using a CASE expression that checks domain-specific rules for each entity. Records that fail any rule are flagged FALSE — they are not deleted, but they are excluded from all downstream Gold tables.

This approach is intentional: it gives data engineers full visibility into quality issues without losing any raw data. The actual validation rules are meaningful and entity-specific:

Silver Table	Example Validation Rules
REGION_MASTER	REGION_CODE not null, REGION_NAME not blank, IS_ACTIVE must be Y or N, end date must not be before start date
TAX_MASTER	TAX_RATE must be between 0 and 1 (e.g., 0.07 = 7%), TAX_INCLUSIVE_FLAG must be Y or N
STORE_MASTER	LATITUDE between -90 and +90, LONGITUDE between -180 and +180, CLOSE_DATE must not be before OPEN_DATE, floor area and rent must be positive
CUSTOMER_MASTER	EMAIL must contain @ and . (basic format check), DATE_OF_BIRTH and REGISTRATION_DATE must not be in the future
SALES_ITEM	QUANTITY must be greater than 0 (you cannot sell zero items), all monetary amounts must be non-negative

Transformation 3: Audit Column Renaming

The audit columns from Bronze are carried forward into Silver, but renamed to reflect their new context:

Bronze Audit Column	Silver Audit Column
__FILE_NAME	__FILE_NAME
__ROW_NUMBER	__SOURCE_ROW_NUMBER
__LOAD_TS	__BRONZE_LOAD_TS

The __BRONZE_LOAD_TS column becomes especially important in the Gold layer — it is used as the basis for the SHA2 surrogate key hash, ensuring each version of a record gets a unique, deterministic key.

GOLD LAYER

Business Data Model — analytics-ready star schema

6. Gold Layer — Business Data Model

The Gold layer is where raw, cleaned data is transformed into a structure optimized for business analysis. Everything in Bronze and Silver exists to serve this layer. The Gold layer answers the question: how do we organize data so business users can ask any question quickly and get a correct answer?

The answer used in this project is a Star Schema — the industry standard for analytical data modeling. In a star schema, you have Dimension tables (the who, what, where, when) surrounding Fact tables (the measurements — what happened, how much). The result looks like a star when drawn as a diagram, hence the name.

6.1 Hash-Based Surrogate Keys — Why SHA2?

Every Gold dimension and fact table uses a hash-based surrogate key instead of an auto-incrementing integer. The key is generated using Snowflake's SHA2 function:

```
SHA2(CONCAT(COALESCE(business_key, ''), COALESCE(TO_VARCHAR(__BRONZE_LOAD_TS, 'YYYY-
MM-DD HH24:MI:SS.FF6'), '')), 256)
```

There are three important reasons for this design choice:

- Deterministic — the same input always produces the same hash. This means if the same record is processed twice (e.g., after a pipeline re-run), it generates the same surrogate key, preventing duplicate rows in the fact table.
- No integer sequences needed — integer auto-increment requires a centralized sequence object. In a distributed, multi-layer pipeline this creates coordination complexity. SHA2 generates the key independently in any layer.
- History-safe — the hash includes the load timestamp, so a new version of the same record (e.g., a customer who changed loyalty tier) gets a different surrogate key. This is the foundation of SCD Type 2 history tracking.

6.2 SCD Type 2 — Tracking Historical Changes

All five-dimension tables implement Slowly Changing Dimension Type 2 (SCD Type 2). This is a dimensional modeling standard for tracking how dimension attributes change over time.

Without SCD Type 2, if a customer upgrades from Silver to Gold loyalty tier, the old value is overwritten and lost. With SCD Type 2, both versions are preserved — you always know what tier a customer was on at the time of any transaction.

Every dimension table contains three SCD Type 2 control columns:

Column	Value
EFFECTIVE_START_TS	Set to __BRONZE_LOAD_TS of that version
EFFECTIVE_END_TS	9999-12-31 23:59:59 for current records
IS_CURRENT	TRUE for the current version

Fact tables join to dimensions using IS_CURRENT = TRUE — ensuring they always join to the correct, current version of each dimension record.

6.3 Dimension Tables

DIM_COUNTRY — Geographic Dimension

DIM_COUNTRY is built by joining four Silver tables together: Country Master, Region Master, Currency Master, and Tax Master. Rather than keeping these as four separate dimension tables (which would force analysts to join four tables for every geographic query), they are consolidated into a single, denormalised geographic dimension.

Source Silver Table	Attributes Contributed to DIM_COUNTRY
COUNTRY_MASTER	COUNTRY_CODE, COUNTRY_NAME, PRIMARY_LANGUAGE, TIMEZONE, MARKET_TIER, ECOMMERCE_SUPPORTED, RETAIL_STORE_SUPPORTED
REGION_MASTER	REGION_CODE, REGION_NAME
CURRENCY_MASTER	CURRENCY_CODE, CURRENCY_NAME, CURRENCY_SYMBOL
TAX_MASTER	TAX_CODE, TAX_TYPE, TAX_RATE, TAX_INCLUSIVE_FLAG

JOIN logic: LEFT JOINs are used from Country to Region, Currency, and Tax — ensuring that country records are not dropped even if a matching currency or tax record is missing.

Current size: 5 rows (5 countries in the dataset)

DIM_PRODUCT — Product Hierarchy Dimension

DIM_PRODUCT flattens the four-level product hierarchy (Category → Family → Model → SKU) into a single, denormalised dimension table. The grain is SKU level — one row per unique SKU. All higher-level attributes (model name, family name, category name, reporting segment) are pulled in via INNER JOINs up the hierarchy chain.

INNER JOINs are used here (not LEFT JOINs) — this is intentional. A SKU that has no matching Model, Family, or Category is considered an orphaned, invalid record and should not appear in the Gold layer.

Current size: 53 rows (53 unique SKUs)

DIM_STORE — Store Dimension

DIM_STORE is loaded directly from Silver STORE_MASTER. It contains full store details including physical attributes (latitude, longitude, floor area, annual rent) and operational attributes (format code: FLAGSHIP/MALL/MINI, lifecycle status, open/close dates). The lat/long columns enable geographic mapping in BI tools.

Current size: 80 rows (80 stores across 5 countries)

DIM_CUSTOMER — Customer Dimension

DIM_CUSTOMER is loaded directly from Silver CUSTOMER_MASTER. It contains the full customer profile including demographics (name, gender, date of birth, address), contact information (email, phone), and segmentation attributes (customer segment: Consumer/Business, loyalty tier: None/Silver/Gold/Platinum). The LOYALTY_TIER column is particularly important for the Semantic Layer, as analysts frequently want to analyze sales performance by loyalty segment.

Current size: 50,000 rows

DIM_DATE — Calendar Dimension

DIM_DATE is different from the other four dimensions — it is not loaded from a source CSV file. Instead, it is derived directly from the SILVER.SALES_HEADER table by extracting all distinct transaction dates and computing every calendar attribute from them.

This approach means the date dimension always covers exactly the date range present in the actual sales data — no more, no less. The table includes calendar attributes (year, quarter, month, week, day), display labels (MONTH_NAME, DAY_NAME, QUARTER_NAME), time intelligence columns (YEAR_MONTH for monthly grouping, YEAR_QUARTER for quarterly grouping), fiscal year/quarter (April–March fiscal calendar), and flags (IS_WEEKEND, IS_HOLIDAY).

> **Note on REFRESH_MODE = FULL for DIM_DATE:** Unlike the other dimensions which use INCREMENTAL refresh, DIM_DATE uses REFRESH_MODE = FULL. This is because it uses SELECT DISTINCT on transaction dates — Snowflake cannot compute incremental changes for DISTINCT aggregations. A full refresh is required each time. The same applies to FACT_SALES_HEADER and FACT_SALES_ITEM.

Current size: 3,443 rows (3,443 unique transaction dates)

6.4 Bridge Table — BRIDGE_PRODUCT_COUNTRY

The BRIDGE_PRODUCT_COUNTRY table solves a specific data modeling challenge: the many-to-many relationship between products and countries.

A single SKU is available in multiple countries. A single country has multiple SKUs available. If you tried to store this directly in DIM_PRODUCT by adding a COUNTRY_CODE column, you would need one row per SKU-country combination — which would explode the product dimension from 53 rows to 212 rows, and all the non-country attributes (model name, family, category) would be repeated for every country. This is called dimension fan-out, and it makes aggregations incorrect.

The correct solution is a Bridge Table — a separate table that only stores the relationship, leaving both dimensions clean:

Column	Purpose
BRIDGE_KEY	Unique hash key for this bridge record (SKU + Country + load timestamp)
PRODUCT_DIM_KEY	Foreign key pointing to the matching row in DIM_PRODUCT
COUNTRY_DIM_KEY	Foreign key pointing to the matching row in DIM_COUNTRY
SKU_CODE	Product business key
COUNTRY_CODE	Country business key
LOCAL_LAUNCH_DATE	Date this SKU became available in this specific country
LOCAL_DISCONTINUE_DATE	Date this SKU was discontinued in this specific country (if applicable)
IS_AVAILABLE	Current availability flag

Current size: 212 rows (53 SKUs × average 4 countries each)

6.5 Fact Tables

FACT_SALES_HEADER — Transaction-Level Fact

The grain of FACT_SALES_HEADER is one row per sales transaction. It captures the financial summary of each transaction — the total gross amount, total discount, total tax, and net total — along with foreign keys to all relevant dimensions.

Column Type	Columns
Fact Key	FACT_SALES_HEADER_KEY (SHA2 hash of TRANSACTION_ID + load timestamp)
Dimension FK	CUSTOMER_DIM_KEY, STORE_DIM_KEY, COUNTRY_DIM_KEY, DATE_DIM_KEY
Degenerate Dims	TRANSACTION_ID, TRANSACTION_NUMBER, CHANNEL_ID (POS/WEB), PAYMENT_METHOD, CURRENCY
Additive Measures	GROSS_AMOUNT, TOTAL_DISCOUNT, TOTAL_TAX, NET_TOTAL
Count Measure	TRANSACTION_COUNT = 1 (constant 1 per row — enables COUNT aggregation in BI tools)

How is COUNTRY_DIM_KEY derived in FACT_SALES_HEADER? The sales transaction in Silver does not directly carry a COUNTRY_DIM_KEY — it only has a STORE_ID. The fact table derives the country by first joining to DIM_STORE (using STORE_ID = STORE_CODE), and then joining to DIM_COUNTRY using the COUNTRY_CODE from the store record. This means country analysis of transactions is always based on where the sale happened, not where the customer lives.

Current size: 86,107 rows

FACT_SALES_ITEM — Line-Item Level Fact

The grain of FACT_SALES_ITEM is one row per product line within a transaction. It captures product-level detail: which SKU was sold, how many units, at what price, with what discount and tax.

A key design decision here: FACT_SALES_ITEM does not independently join to all dimensions. Instead, it joins to FACT_SALES_HEADER and inherits the dimension keys from the header fact. This means the item fact only needs to add PRODUCT_DIM_KEY on top of what the header already provides — all other foreign keys (customer, store, country, date) are pulled from the header row.

Column Type	Columns
Fact Key	FACT_SALES_ITEM_KEY (SHA2 hash of TRANSACTION_LINE_ID + load timestamp)
Parent Fact FK	FACT_SALES_HEADER_KEY (links back to the parent transaction)
Inherited FK	CUSTOMER_DIM_KEY, STORE_DIM_KEY, COUNTRY_DIM_KEY, DATE_DIM_KEY (inherited from header)
Product FK	PRODUCT_DIM_KEY (the only new dimension key added at item level)
Degenerate Dims	TRANSACTION_LINE_ID, TRANSACTION_ID, CHANNEL_ID, CURRENCY
Additive Measures	QUANTITY, UNIT_PRICE, DISCOUNT_AMOUNT, TAX_AMOUNT, LINE_TOTAL
Count Measure	LINE_COUNT = 1 (constant 1 per row)

Why two separate fact tables instead of one? Because they serve different analytical purposes. FACT_SALES_HEADER is used for transaction-level analysis — revenue by region, channel performance, customer spending. FACT_SALES_ITEM is used for product-level analysis — units sold by SKU, revenue by category, top products. Combining them into one table would either require duplicating the header measures across every line item, or lose the ability to drill into product-level detail. Two tables is the correct modeling decision.

Current size: 86,107 rows (1 line item per transaction in this dataset)

6.6 Aggregated Fact Tables — Pre-Computed Summaries

The three aggregated fact tables sit on top of FACT_SALES_HEADER and DIM tables in the Gold layer. They pre-compute summarized metrics at different time grains so that dashboard queries do not need to scan and aggregate 86,107 transaction rows every time a user opens a report.

Table	Time Grain	Target Lag / Refresh
FACT_SALES_DAILY	Day-level	5 minutes / INCREMENTAL
FACT_SALES_WEEKLY	Week-level	7 days / INCREMENTAL
FACT_SALES_MONTHLY	Month-level	30 days / INCREMENTAL

The lag settings are deliberately different and reflect the actual use case of each table:

- FACT_SALES_DAILY refreshes every 5 minutes — daily dashboards need near-real-time data.
- FACT_SALES_WEEKLY refreshes every 7 days — weekly reports are produced once a week, so more frequent refresh would waste compute.
- FACT_SALES_MONTHLY refreshes every 30 days — monthly executive reports are produced once a month.

All three aggregated tables are INCREMENTAL Dynamic Tables, joining FACT_SALES_HEADER to DIM_DATE and DIM_COUNTRY (and DIM_STORE for the daily table). They compute standard measures including transaction count, gross amount, total discount, total tax, net sales, average transaction value, unique customer count, active store count (weekly/monthly), and selling days (weekly/monthly).

The monthly table additionally includes fiscal year and fiscal quarter columns (April–March fiscal calendar), making it directly usable for executive financial reporting without any additional calculation.

6.7 Informational Constraints — PK and FK Definitions

Once all Gold tables are created, Primary Key and Foreign Key constraints are explicitly defined — but with a crucial Snowflake-specific setting: NOT ENFORCED.

> **What does NOT ENFORCED mean?** In traditional databases (SQL Server, Oracle), FK constraints are enforced at write time — the database will reject any row that violates a relationship. In Snowflake, constraints are informational only. They are NOT checked when data is inserted. Instead, they serve three purposes: (1) they document the intended relationships between tables, (2) they provide hints to Snowflake's query optimizer, and (3) they enable BI tools and the Cortex Analyst Semantic Layer to automatically discover table relationships without being told manually.

Constraints defined in this project:

- Primary Keys: All 5 dimension tables + both fact tables — 7 PK constraints total
- Foreign Keys on FACT_SALES_HEADER: → DIM_CUSTOMER, → DIM_STORE, → DIM_COUNTRY, → DIM_DATE
- Foreign Keys on FACT_SALES_ITEM: → FACT_SALES_HEADER (parent fact), → DIM_PRODUCT

SEMANTIC LAYER

Natural Language Analytics — talk to your data in plain English

7. Semantic Layer — Cortex Analyst

7.1 What is a Semantic Layer and Why Does It Matter?

Even after the Gold layer is fully built, there is still a barrier between business users and the data: SQL. An analyst who wants to know 'Which region had the highest net sales last quarter?' needs to know the table names, the correct join paths between FACT_SALES_HEADER, DIM_COUNTRY, and DIM_DATE, the right column names, and how to write a GROUP BY aggregation.

A Semantic Layer removes this barrier entirely. It is a translation layer — a business-friendly vocabulary built on top of the Gold schema — those maps plain English terms and questions to the correct SQL automatically.

Snowflake's implementation of this is called Cortex Analyst, and it is powered by a YAML-based Semantic Model file that describes the Gold layer in business terms.

> **Simple analogy:** Think of the Semantic Layer as a GPS for your data. Without GPS, you need to know every road name, every turn, every route. With GPS, you just say 'take me to the nearest coffee shop' and it figures out the route for you. The Semantic Layer is the GPS for business users navigating your data warehouse.

7.2 The Cortex Analyst Prompt

The prompt used to generate the Semantic Model with Cortex Code was:

> **Cortex Code Prompt Used:** "Create a semantic model by using all the fact/dim tables that exist in gold layer and enable the snowflake analyst, so I can talk to the data using natural language and ask natural questions on my sales data"

Cortex Code generated a complete YAML semantic model file (sales_semantic_model.yaml) covering all Gold tables, which was then uploaded to a Snowflake stage to enable Cortex Analyst.

7.3 What is Defined in the Semantic Model?

The semantic model (named sales_analytics) defines four key components for each Gold table:

- Tables — which Gold tables exist and what they represent in plain English
- Dimensions — the descriptive columns users can filter and group by, with plain English names and synonyms
- Metrics — pre-defined calculated measures (SUM, COUNT, AVG) with synonyms so multiple phrasings resolve to the same calculation
- Relationships — the join paths between tables, so Cortex Analyst knows how to connect them

7.4 Tables in the Semantic Model

Table Name	Description in Semantic Model
dim_customer	Customer master data with demographics, location, and loyalty information
dim_product	Product hierarchy with category, family, model, and SKU details
dim_store	Store locations with format, size, and geographic attributes
dim_country	Country reference with region, currency, and tax information
dim_date	Calendar date dimension for time-based analysis
fact_sales_header	Sales transactions at header level — one row per transaction with totals
fact_sales_item	Sales transactions at line-item level — one row per product sold

7.5 Key Metrics Defined

Metrics are the core of the Semantic Layer — they are the pre-defined calculations that Cortex Analyst resolves when a user asks a quantitative question. Each metric has a name, a SQL expression, and multiple synonyms so different phrasings of the same question resolve correctly:

Metric Name	SQL Expression
total_net_sales	SUM(NET_TOTAL)
total_gross_sales	SUM(GROSS_AMOUNT)
total_transactions	COUNT(DISTINCT TRANSACTION_ID)
average_transaction_value	AVG(NET_TOTAL)
total_units_sold	SUM(QUANTITY)
total_line_revenue	SUM(LINE_TOTAL)
average_unit_price	AVG(UNIT_PRICE)
store_count	COUNT(DISTINCT STORE_CODE)
customer_count	COUNT(DISTINCT CUSTOMER_ID)
product_count	COUNT(DISTINCT SKU_CODE)

7.6 Synonyms and Enumerations — Making NLP Reliable

Two features of the semantic model make natural language query resolution significantly more reliable:

Synonyms

Key dimensions are given multiple aliases so that different phrasings of the same concept resolve to the same column. For example, the CUSTOMER_SEGMENT column is mapped to synonyms including 'segment' and 'market segment'. The LOYALTY_TIER column maps to 'loyalty level' and 'membership tier'. The REGION_NAME column maps to 'region' and 'geographic region'.

Enumerations (is_enum with sample_values)

For categorical columns, the semantic model specifies that they are enumerations and provides sample values. For example:

- LOYALTY_TIER: sample values None, Silver, Gold, Platinum
- STORE FORMAT_CODE: sample values MALL, MINI, FLAGSHIP
- CHANNEL_ID: sample values POS, WEB
- CATEGORY_NAME: sample values iPhone, iPad, Mac, Apple Watch, AirPods, Apple TV, Accessories, AppleCare, Services

Providing these sample values helps Cortex Analyst understand the valid values for each field, which improves the accuracy of SQL generated from natural language queries that reference these values (e.g., 'show me sales from flagship stores' correctly resolves LIFECYCLE_STATUS = 'FLAGSHIP').

7.7 Relationships Defined in the Semantic Model

The semantic model explicitly defines all join paths between the Gold tables. This tells Cortex Analyst which columns to use when connecting tables — removing any ambiguity:

Relationship	Left Table
header_to_customer	fact_sales_header
header_to_store	fact_sales_header
header_to_country	fact_sales_header
header_to_date	fact_sales_header
item_to_header	fact_sales_item
item_to_product	fact_sales_item
item_to_customer	fact_sales_item
item_to_store	fact_sales_item
item_to_country	fact_sales_item
item_to_date	fact_sales_item

7.8 Pre-Verified (Golden) Queries

The semantic model includes a set of verified queries — SQL statements for the most common business questions that have been manually written and validated. Cortex Analyst uses these as anchors when resolving similar natural language queries, significantly improving accuracy for questions it has seen variations of before:

- "What are total sales by region?"
- "What are the top 10 products by revenue?"
- "Show me the monthly sales trend"
- "What are sales by customer loyalty tier?"
- "Which stores have the highest sales?"
- "What are sales by product category?"
- "Compare sales between POS and Web channels"
- "What are sales by quarter?"

7.9 The End Result — Talking to Your Data

With the Semantic Layer in place and Cortex Analyst enabled, a business user can open Snowflake and type questions like these — and get instant, accurate answers without writing a single line of SQL:

User Types This...	Cortex Analyst Does This...
"Total net sales by region"	Joins FACT_SALES_HEADER → DIM_COUNTRY, groups by REGION_NAME, sums NET_TOTAL
"Top 10 products by revenue this year"	Joins FACT_SALES_ITEM → DIM_PRODUCT → DIM_DATE, filters current year, groups by MODEL_NAME, sums LINE_TOTAL, limits 10
"How many customers bought from flagship stores?"	Joins FACT_SALES_HEADER → DIM_STORE, filters FORMAT_CODE = 'FLAGSHIP', counts distinct CUSTOMER_DIM_KEY
"Monthly sales trend for platinum customers"	Joins FACT_SALES_HEADER → DIM_CUSTOMER → DIM_DATE, filters LOYALTY_TIER = 'Platinum', groups by YEAR_MONTH, sums NET_TOTAL

8. End-to-End Data Flow Summary

The complete data journey from raw file to natural language answer:

Step	What Happens
1	13 CSV files uploaded to Internal Stage (BRONZE schema) via Snowsight WebUI
2	COPY INTO command loads each file into its corresponding Bronze raw table, adding __FILE_NAME, __ROW_NUMBER, __LOAD_TS metadata columns
3	Silver Dynamic Tables detect new Bronze data and auto-refresh: deduplication via QUALIFY ROW_NUMBER(), data quality check via IS_VALID_RECORD flag, audit column renaming
4	Gold Dimension Dynamic Tables refresh from Silver: DIM_COUNTRY joins 4 tables, DIM_PRODUCT flattens 4-level hierarchy, DIM_STORE/CUSTOMER loaded directly, DIM_DATE derived from transaction dates
5	FACT_SALES_HEADER built from Silver Sales Header + Gold dimension keys. FACT_SALES_ITEM built from Silver Sales Item + FACT_SALES_HEADER (inheriting dim keys) + DIM_PRODUCT
6	BRIDGE_PRODUCT_COUNTRY resolves SKU-Country many-to-many relationship
7	3 Aggregated fact tables (Daily/Weekly/Monthly) pre-compute summary metrics at different time grains with different TARGET_LAG settings
8	Informational PK/FK constraints defined (NOT ENFORCED) to document relationships for BI tools and Cortex Analyst
9	YAML Semantic Model (sales_semantic_model.yaml) uploaded to Snowflake stage. Defines 7 tables, 10 metrics with synonyms, 10 relationships, 8 verified queries
10	Business users query the Gold layer using natural language via Cortex Analyst — no SQL required

9. Key Engineering Design Decisions

Every significant design decision in this project was made for a specific reason. This section documents the rationale behind the most important choices:

Design Decision	Rationale
Dynamic Tables instead of Streams + Tasks	Simpler architecture — no manual stream management or task chaining. Built-in incremental processing. Lower compute cost. Less code to maintain.
TRANSIENT databases for Dev and QA	Eliminates Snowflake Fail-safe storage charges for non-production environments. Dev/QA data is always reproducible so fail-safe protection is not needed.
7-day Time Travel for Production	Maximum recovery window on Snowflake Standard Edition. Covers the typical incident detection and response window for production data issues.
SHA2 hash surrogate keys	Deterministic (no duplicates on re-run), no sequence objects needed, naturally supports SCD Type 2 history (different load timestamp = different key).
SCD Type 2 on all 5 dimensions	Preserves historical attribute states. Essential for accurate time-based analysis — e.g., tracking what loyalty tier a customer had at time of purchase.
DIM_COUNTRY consolidates 4 Silver tables	Avoids forcing analysts to join 4 separate tables for geographic queries. Denormalised design is standard in star schemas.
INNER JOINs in DIM_PRODUCT hierarchy	Orphaned SKUs (missing model/family/category) are invalid data and should not appear in Gold. INNER JOIN enforces referential integrity during modeling.
LEFT JOINs in DIM_COUNTRY	Country records should appear even if currency or tax data is missing. LEFT JOIN ensures no countries are silently dropped due to missing reference data.
FACT_SALES_ITEM inherits FK from header fact	Avoids redundant joins to all dimensions at item level. Header fact already has the correct dimension keys — item fact reuses them via FACT_SALES_HEADER_KEY.
COUNTRY derived via STORE in header fact	Transactions do not carry a country code — they carry a store ID. Country is correctly derived from where the sale happened (the store), not customer address.
DIM_DATE uses REFRESH_MODE = FULL	SELECT DISTINCT cannot be processed incrementally by Snowflake. Full refresh is required. Same applies to both fact tables.
Different TARGET_LAG on aggregated tables	Aligns refresh frequency with actual business use — 5 min for daily operational dashboards, 7 days for weekly reports, 30 days for monthly executive reports.
Bridge table for Product-Country	Many-to-many resolved correctly — no dimension fan-out, no measure double-counting. Standard bridge table pattern.
NOT ENFORCED constraints	Documents relationships for BI tools and Cortex Analyst without runtime enforcement overhead. Standard Snowflake best practice for analytical schemas.

COMMENT on every object and column	Self-documenting schema — critical for team collaboration, BI tool metadata discovery, and Semantic Layer accuracy.
COMMON schema for shared utilities	Single maintenance point for CSV file format and other reusable objects. Change once, applies everywhere.

10. What This Project Demonstrates

10.1 Technical Skills

- Snowflake-native data engineering — Internal Stage, COPY INTO, INFER_SCHEMA, Dynamic Tables, Time Travel, informational constraints
- Advanced SQL — QUALIFY with ROW_NUMBER() window function for deduplication, SHA2 hash key generation, CASE-based data quality flags, multi-table JOIN hierarchies
- Dimensional data modeling — Star schema design, SCD Type 2 implementation, bridge table for many-to-many, degenerate dimensions, fact-to-fact relationships
- AI-assisted development — Snowflake Cortex Code (CoCo) used throughout for DDL generation, transformation logic, and semantic model scaffolding
- Semantic layer design — YAML-based Cortex Analyst model with metrics, synonyms, enum sample values, relationship definitions, and verified golden queries
- Environment management — Dev/QA/Prod promotion strategy, transient vs permanent database strategy, schema separation
- Data quality engineering — entity-specific validation rules, IS_VALID_RECORD flagging, audit metadata columns, data lineage tracing

10.2 Professional Practices Applied

- Every database object has a meaningful COMMENT — self-documenting schema
- Transient tables in non-production environments — cost optimization
- Production-grade metadata tracking (__FILE_NAME, __ROW_NUMBER, __LOAD_TS) — full audit trail
- Informational FK/PK constraints — schema documentation for BI tools
- Bridge table for many-to-many — correct modeling, no fan-out
- Separate fact tables by grain — no measure duplication or granularity loss
- Pre-computed aggregations with different refresh lags — performance optimization aligned to business use

10.3 One-Line Summary

A fully automated, AI-assisted, medallion-based Snowflake data platform with star schema modeling and natural language analytics capability.